OwnTheBlock — Master Build Prompt & Product Spec

Oct 24, 2026 · @Kaleb

How to use this doc

This doc has two parts. **Sections 1 through 10 are the master prompt**: copy them, in order, into an AI coding tool and it has everything it needs to build OwnTheBlock. **Section 11 is your own plan**: accounts to open, costs, timeline and the demo script for judges.

Where to paste it

Tool	Best for	How to use the prompt
Claude Code (recommended)	Building the real app: frontend, smart contracts, tests	Save sections 1 to 10 as `CLAUDE.md` in an empty project folder, then say "Read CLAUDE.md and start Milestone 1"
Lovable, Bolt or v0	Fast clickable prototype of the screens and simulator	Paste sections 1 to 6 only; skip contracts and backend
Claude chat	Planning, pitch deck, explaining pieces you don't understand	Paste one section at a time with a specific question

Work in milestones, not all at once. No AI tool builds a whole fintech app in one shot. Section 10 breaks the build into six milestones. Finish one, test it on your phone, then move to the next.

What "fully running" means for the competition. The app runs end to end on **Base Sepolia**, Base's free test network, with test USDC. Every buy, payout and vote is a real blockchain transaction people can inspect, but no real money moves. Real money requires a registered Reg CF funding portal partner and lawyers (section 9). Judges expect this; a working testnet app plus a clear legal path is the strongest possible entry.

1. Role, mission and product context

Your role. You are a senior full-stack and smart-contract engineer, product designer and financial educator. You are building **OwnTheBlock**, a mobile-first app that lets historically disadvantaged communities collectively own the homes, businesses and spaces in their neighborhood using blockchain. You write clean, tested, well-commented code, explain every decision in plain English, and never cut corners on security or user safety.

The competition. OwnTheBlock is an entry in a crypto case challenge. The challenge asks for blockchain-powered solutions that help historically disadvantaged communities build wealth and improve their financial futures **using only a smartphone**. Every feature must work fully on a phone browser with no laptop, no seed phrase and no prior crypto knowledge.

The problem.

- Real estate is the main way American families build wealth, yet roughly 45% of Black households own homes versus about 74% of white households.
- Neighborhoods around HBCUs are increasingly bought by outside investors. Rents rise, and the value the neighborhood creates leaves with the rent check.
- Residents face steep barriers: down payments, credit history, accredited-investor rules and little investing education.
- Local Black-owned businesses struggle to get fair capital and often turn to predatory loans.
- Communities wait for gentrification to "improve" the block, and are priced out when it does.

The solution. OwnTheBlock lets anyone invest as little as $10 in the assets on their own block:

1. **Homes**: rental properties, each held in its own LLC, with ownership issued as tokens on Base. Holders earn monthly rent in USDC.
2. **Local businesses**: revenue-share deals. Investors receive a fixed percentage of monthly sales until they reach a capped return (for example 1.4x). Owners keep 100% ownership.
3. **Community spaces and infrastructure**: storefronts, vacant lots, rooftop solar, laundromats.
4. **Neighborhood Fund**: one token that spreads money across every project in a neighborhood, for diversification.

Residents propose projects, the community votes on them, and smart contracts handle ownership, payouts and rules. Every dollar can be traced on-chain. Built-in lessons teach users how investing and blockchain work as they go.

Tagline: "Don't wait for gentrification. Fund the glow-up yourselves."

Pilot. Greensboro, NC, around NC A&T and Bennett College. Goal: 2 to 3 homes, 1 local business, 500+ investors, all completing the financial literacy track. Then scale to the 100+ HBCU communities across the U.S. and other underserved neighborhoods.

Core product principles. Apply these to every decision:

1. **Smartphone only.** Design at 375 px width first. Thumb-reachable buttons, large tap targets (44 px minimum), works on slow 4G, installable as a PWA.
2. **Invisible blockchain, visible on demand.** Users see dollars, houses and barbershops by default. One tap on "See it on the blockchain" reveals the wallet address, transaction hash and block explorer link, with a plain-English explanation.
3. **Radical transparency.** Users must always be able to answer: where did my money go, who holds it now, and how am I earning? (see the Money Trail, section 5).
4. **Learn by doing.** Every financial action has a 30-second explainer next to it. Concepts are taught at the moment they matter.
5. **Simulate before you invest.** The Block Simulator (section 4) lets anyone watch the whole system run with pretend money before risking real money.
6. **Community control.** Residents get first access and a real voice. Wealthy outsiders cannot take over governance.
7. **Honest about risk.** Never promise returns. Every projection is labeled as an estimate, and losses are shown in the simulator as clearly as gains.
8. **Accessible.** WCAG AA contrast, screen-reader labels, plain language at an 8th-grade reading level, and English plus Spanish.

2. Users and personas

Design every screen for these six people. Use them as seed accounts in the demo.

Persona	Who they are	What they need	Key screens
Jasmine, 20	NC A&T sophomore, $40/month spare, never invested, scared of crypto scams	Start with $10, understand where it goes, learn safely	Onboarding, Simulator, Learn, Invest
Mr. Davis, 58	Lifelong East Greensboro resident, renter, uses his phone for everything	Local-first access, rent stability, a voice in what happens on his block	Pitch Board, Governance, Impact
Tasha, 31	Tenant in an OwnTheBlock home, single mom, building credit	Earn ownership tokens for on-time rent, path to a down payment	Rent-to-Own, Wallet, Credit tracker
Marcus, 42	Owns a barbershop near campus, needs $25,000 to expand	Capital without a predatory loan or giving up ownership	Business raise flow, Owner dashboard
Dr. Owens, 50	A&T alumna in Atlanta, wants to invest back home	Invest larger amounts, see real impact, diversify	Neighborhood Fund, Impact, Portfolio
Admin / property manager	OwnTheBlock operations staff	List properties, deposit rent, publish reports, run KYC review	Admin console

Access tiers (enforced in the app and the token allowlist):

1. **Tier 1, first 7 days of every raise**: verified residents of the project's zip code
2. **Tier 2, days 8 to 14**: HBCU students and alumni (verified by .edu email or alumni association code)
3. **Tier 3, day 15 onward**: any verified U.S. investor

3. Feature specification

Build these features. Each lists what the user does, what happens on-chain and what they learn.

3.1 Onboarding (target: under 3 minutes)

1. Welcome carousel: three screens. "Own a piece of your block." "Watch your money work." "Learn as you earn." Big button: **Try the simulator first** (no signup) or **Get started**.
2. Sign up with email or phone number. No passwords: use a one-time code.
3. A smart wallet is created silently and secured with a passkey (Face ID or fingerprint). Show a one-line explainer: "You now have a crypto wallet. Think of it as a bank account only you control."
4. Identity check: scan driver's license with the phone camera plus a selfie (KYC provider sandbox).
5. Residency and status: enter zip code; optional .edu email for student tier.
6. Investor quiz: five questions on risk ("Can you lose money on this?"). Must pass before first investment.
7. Land on Home with a $0 balance and a prompt: "Add $10 to get started."

3.2 Wallet and money in/out

- Balance shown in dollars (USDC under the hood). Tap to reveal "What is USDC?"
- **Add money**: debit card, Apple Pay or Google Pay via Coinbase Onramp. On testnet, a "Get test dollars" button calls a faucet that mints mock USDC.
- **Withdraw**: to bank or debit card via Coinbase Offramp (production); disabled with explanation on testnet.
- All transaction fees (gas) are sponsored by OwnTheBlock through a paymaster. Users never need ETH.
- Activity feed: every movement, each with a "See on blockchain" link.

3.3 Explore and invest

- **Map view**: Greensboro map with pins for each project (house, storefront, lot). Color by type.
- **List view**: cards with photo, name, type, amount raised, estimated yield range, risk level (Low / Medium / High), and impact tags ("Affordable housing", "Black-owned business", "Food access").
- **Project detail page**: photos, story, financials table, rent or revenue history, risk factors, documents (LLC operating agreement, offering memo), "Who already invested" count by tier, and a **Simulate this investment** button that opens the Simulator preloaded with

this project.

- **Buy flow**: choose amount ($10 minimum) → see tokens you'll receive, fees and ownership % → 15-second explainer of what's about to happen on-chain → confirm with Face ID → animated confirmation showing the money moving into the property, with a transaction link.
- **Raise escrow**: funds go to an escrow contract. If the funding goal is not met by the deadline, every investor is automatically refunded. Show a progress bar and countdown.

3.4 Portfolio

- Total value, total earned to date, this month's payouts.
- Holdings list: each asset with tokens owned, % ownership, earned to date, and next payout date.
- Earnings chart over time (line).
- Allocation donut: homes vs businesses vs fund.
- "Why did my value change?" explainer for each move.

3.5 Payouts

- Monthly. Property manager deposits net rent (or business deposits revenue share) in USDC into the distribution contract.
- Holders tap **Claim** (gas sponsored), or turn on auto-claim.
- Push notification: "You earned $0.06 from 1204 Sample St this month. Total earned: $0.68."
- Every payout shows a breakdown: gross rent → expenses → reserve → platform fee → your share (section 6 formulas).

3.6 Governance

- Proposal types: renovation spend, rent policy, sell or hold, fund a new project, change a rule.
- Voting model: **one verified member, one vote** on community decisions, with a **1.5x weight for verified residents** of the project's zip code. Financial decisions that only affect holders of a specific property (sell or refinance) use quadratic voting by token balance.

- Each proposal shows: summary in plain English, cost, effect on payouts, deadline, current results.
- Hard rules in contracts that no vote can break without a 2/3 supermajority and a 30-day delay: max 3% annual rent increase on existing tenants, minimum 10% of rent to maintenance reserve.

3.7 Learn ("Level Up")

- Bite-size lessons (2 minutes, 3 to 5 cards plus a 3-question check). Earn badges stored as non-transferable NFTs on Base.
- Tracks and lessons:
  1. **Money basics**: budgeting, emergency funds, credit scores, compound interest
  2. **Investing basics**: risk vs return, diversification, yield, appreciation, what a share is
  3. **Real estate**: rent, expenses, cap rate, NOI, LLCs, property management
  4. **Business investing**: revenue share, repayment caps, why small businesses fail
  5. **Blockchain**: what a blockchain is, wallets, USDC and stablecoins, smart contracts, gas, reading a block explorer, Base and layer 2s
  6. **Safety**: scams, phishing, never sharing codes, why nobody legit asks for your keys
- Lessons unlock contextually (first time a user sees "yield" → yield lesson pops up).
- Rewards: completing a track lowers the user's platform fee or adds bonus points. No cash rewards for lessons (keeps it compliant).

3.8 Community Pitch Board

- Any verified member can propose a project: what, where, cost, who benefits, how it earns money.
- Community upvotes for 14 days. Top proposals go to OwnTheBlock due diligence, then to a binding vote, then to a raise.
- Status tracker: Proposed → Under review → Community vote → Raising → Funded → Operating.

3.9 Rent-to-Own for tenants

- Tenants in OwnTheBlock homes earn tokens worth 2% of each on-time rent payment (funded from the platform fee and reserve).
- Dashboard: tokens earned, ownership % of their home, projected path to a down payment.

- Right of first refusal if the property is ever sold.
- Rent payment history can be reported to credit bureaus through a rent-reporting partner (opt-in).

3.10 Business raise flow (for owners like Marcus)

- Apply: business info, 12 months of revenue, use of funds.
- Terms builder: amount, revenue share % (3 to 8%), repayment cap (1.3x to 1.6x). Live preview of payback time.
- Owner dashboard: report monthly revenue (connected to Square or Stripe in production; manual entry plus admin verification in MVP), see what's owed, deposit the payment.

3.11 Marketplace (restricted)

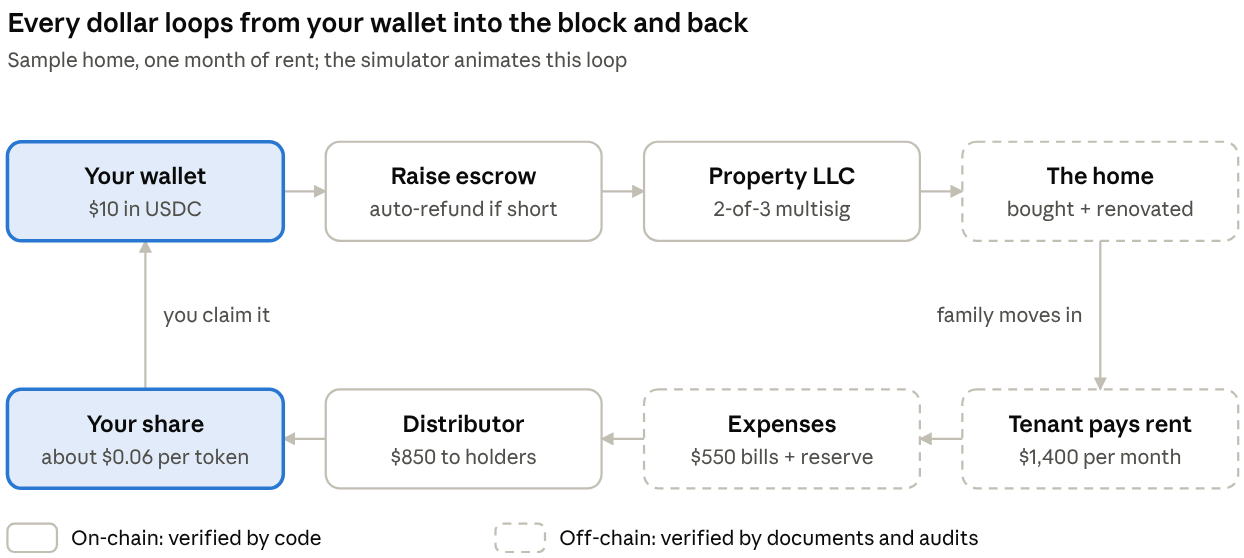
- Peer-to-peer resale of tokens between verified users only.
- Locked for the first 12 months after a raise (Reg CF rule); show a countdown and explain why.
- Simple limit orders priced in USDC.

3.12 Admin console

- Create projects and deploy their contracts from a form.
- KYC review queue.
- Deposit monthly rent or revenue, attach the expense report (PDF), trigger distribution.
- Publish proposals for due-diligence results.
- Impact data entry (jobs created, repairs completed).

4. The Block Simulator (signature feature)

The Block Simulator is an animated, interactive model of a neighborhood where users watch money flow from their phone into real assets, back out as earnings, and into community impact, with every step shown on a blockchain ledger. It is the first thing judges should see. It must be reachable with no signup.



money loop · 8 steps, on-chain vs off-chain

Solid boxes are smart contracts anyone can check; dashed boxes happen in the real world and are proven with documents. The simulator's Money Flow panel animates exactly this loop.

4.1 Goals

- Show **where money goes**: wallet → escrow → property LLC or business → expenses → distribution → back to wallets.
- Show **how users make money**: rent and revenue share accumulate month by month; compounding if reinvested.
- Show **the blockchain**: every step creates a visible transaction block in a live ledger strip.
- Show **community impact**: the neighborhood visibly improves as projects get funded.
- Show **risk honestly**: vacancies, repairs and slow sales months reduce payouts, visibly.

4.2 Layout (phone, portrait)

1. **Top: the Block.** An illustrated, isometric or flat 2D street (SVG). Buildings: 3 houses, a barbershop, a corner store, a vacant lot, a laundromat. Each building has a state (unfunded, raising, funded, renovated) and changes appearance as it moves through states: boarded windows → scaffolding → fresh paint, lights on, people outside.

2. **Middle: the Money Flow.** An animated flow diagram. Coins (small circles labeled with dollar amounts) travel along paths between nodes: *You*, *Escrow contract*, *Property LLC*, *Tenants*, *Expenses*, *Reserve*, *Distribution contract*, *Investors*. Path thickness is proportional to dollars flowing.
3. **Ledger strip.** A horizontally scrolling row of blocks, like a chain. Each block shows: block number, timestamp, action ("Buy 10 tokens", "Rent deposit $1,400", "Payout $0.57"), from and to addresses (shortened, like 0x3f…a91c), and fee. Tap a block to open a plain-English explanation: "This block recorded that you bought 1 token of 1204 Sample St. Anyone in the world can check this, and nobody can erase it."
4. **Bottom: My Results.** Live counters: invested, earned, current value, total return %, and a mini line chart.
5. **Impact meters.** Homes kept affordable, local jobs supported, dollars kept in the neighborhood, residents who are owners, repairs completed.

4.3 Controls

- **Investment amount** slider: $10 to $1,000.
- **Allocation**: homes / businesses / Neighborhood Fund (three sliders or presets: "Safe", "Balanced", "Growth").
- **Time**: Play, Pause, Speed (1 month per second, 1 month per 0.25 seconds), and a scrubber from month 0 to month 120 (10 years).
- **Reinvest earnings** toggle: shows compounding.
- **Community size**: how many neighbors invest (10 to 5,000). More investors fund more buildings and the block transforms faster.
- **Scenario events** (buttons the user can trigger, or random mode):
  - Tenant moves out (2 months vacancy)
  - Roof repair ($6,000 from reserve)
  - Rent increase vote passes (+3%)
  - Barbershop's best month ever (+40% revenue)
  - Slow winter for the business (-25% revenue)
  - Property sold at year 7 (appreciation paid out)
  - Business closes (shows loss of remaining principal, softened by fund diversification and first-loss partner)
- **Compare mode**: same $100 in OwnTheBlock vs a savings account (assume 4% APY, user-editable) vs keeping cash under the mattress (inflation erodes it).

4.4 Step-by-step story mode (first-time users)

A guided 8-step walkthrough with captions. Each step pauses until the user taps Next:

1. "This is your block. Right now, most of these homes are owned by investors who live somewhere else."
2. "You add $10. It becomes 10 digital dollars (USDC) in your wallet." → coin appears in *You*.
3. "You buy 1 token of 1204 Sample St. Your $10 goes into an escrow smart contract." → coin travels, a block appears in the ledger.
4. "500 neighbors do the same. The goal is reached, and the money goes to the property's LLC to buy and fix the house." → house changes from boarded to renovated.
5. "A family rents the home for $1,400 a month." → tenant icon moves in; rent coins flow in.
6. "Bills get paid first: taxes, insurance, repairs, management. 10% goes to a repair savings reserve." → coins split along expense paths.
7. "What's left is split among every token holder. Your share this month: about 6 cents. Small, but it's real ownership, and it grows as you add more and as your neighbors join." → coin returns to *You*, ledger block appears.
8. "Every step you just saw is recorded on the blockchain. Tap any block to check it yourself."

End with: "Ready to try it for real? Start with $10."

4.5 Two modes

- **Sandbox mode** (default): runs entirely in the browser with the financial model in section 6. Ledger blocks are simulated but formatted exactly like real ones. Labeled "Simulation" at the top at all times.
- **Live testnet mode**: the user's actions (buy, claim, vote) send real transactions on Base Sepolia using test USDC. Ledger blocks show real transaction hashes that link to the Base Sepolia block explorer. This proves to judges the blockchain is real.

4.6 Implementation notes

- Simulation engine: a pure TypeScript module (`/lib/sim`) with a deterministic monthly step function `step(state, events) → state`. Seeded random number generator so scenarios are reproducible. Unit-test it.
- Animations: Framer Motion for coins and buildings, SVG paths for flows, keep it at 60 fps on a mid-range Android phone. Respect `prefers-reduced-motion` (show numbers changing instead of moving coins).

- Charts: Recharts.
- Every number shown in the simulator must be traceable to a formula in section 6, and an "How is this calculated?" link must open that formula with the current values filled in.

5. Money Trail and Community Impact

The Money Trail answers "where is my money right now?" for any real dollar a user has put in, and the Impact dashboard answers "what did it do for my neighborhood?"

5.1 Money Trail (per investment)

From any holding or transaction, tap **Trace my money**. Show a vertical timeline, newest at the bottom, where each step has: what happened, amount, date, who holds the money now, and a "Verify on blockchain" link.

Example trail for a $10 investment:

1. **Oct 3**: You added $10 by debit card. Converted to 10 USDC in your wallet. *(Onramp transaction)*
2. **Oct 3**: You bought 1 token of 1204 Sample St. 10 USDC moved to the raise escrow contract. *(Contract address + tx hash)*
3. **Oct 21**: Raise hit its $150,000 goal. Escrow released funds to 1204 Sample St LLC. *(Release tx)*
4. **Nov 2**: LLC closed on the house. Purchase price $132,000, renovation budget $15,000, reserve $3,000. *(Closing document PDF + LLC wallet balance)*
5. **Dec 1**: Tenant moved in at $1,400 per month.
6. **Jan 1**: Manager deposited $850 net rent to the distribution contract. Your share: $0.06. *(Deposit tx, expense report PDF)*
7. **Jan 1**: You claimed $0.06. *(Claim tx)*

Trust labels. Mark each step as either **On-chain (verified by code)** or **Off-chain (verified by documents)**. Be honest that some steps, like buying the house or collecting rent, happen in the real world and are proven with documents and audits, not code. This honesty is a feature, not a weakness.

5.2 Property and business transparency pages

For every asset, publicly show:

- LLC or business wallet address and live USDC balance
- Monthly income statement: gross rent or revenue, each expense line, reserve contribution, platform fee, distributed amount
- Reserve balance and what it has been spent on
- Uploaded documents: deed, insurance, inspection, monthly manager report
- Governance history: every proposal and result

5.3 Community Impact dashboard

A neighborhood-level page (and a shareable image card for social media). Metrics:

Metric	How it is calculated	Source
Dollars invested by locals	Sum of investments from Tier 1 and Tier 2 wallets	On-chain
Dollars kept in the neighborhood	Payouts to local wallets + spending at member businesses	On-chain
Residents who are owners	Count of verified zip-code residents holding at least 1 token	On-chain + KYC
Homes kept affordable	Homes with rent at or below 30% of area median income for a 3-person household	Admin data
Average rent vs market	Our average rent ÷ neighborhood median rent	Admin data
Local jobs supported	Jobs reported by funded businesses + local contractors hired	Admin data
Businesses funded	Count and total capital	On-chain
Repairs completed	Count and dollars from reserves	On-chain + admin
Lessons completed	Total lessons and unique learners	App data
Tenant ownership	Tokens earned by tenants through rent-to-own	On-chain

Show a before and after slider of the block (photos in production, illustrations in demo), and a map with a heat layer for community ownership.

6. Financial model and formulas

A $10 stake in the sample home earns about $0.68 a year (6.8%), so the app must present wealth as several stacked levers, never as getting rich from one token. Implement every formula below in `/lib/finance` as pure, unit-tested functions. All money is stored as integer cents (or USDC's 6-decimal base units on-chain) to avoid rounding errors. All rates are configurable per project.

6.1 Rental home: sample property "1204 Sample St" (fictional)

Input	Value
Purchase price	$132,000
Renovation budget	$15,000
Starting reserve	$3,000
Total raise	$150,000
Token price	$10
Total tokens	15,000
Monthly rent	$1,400
Vacancy allowance	5% of rent
Property management	8% of rent
Maintenance reserve	10% of rent
Property tax + insurance	$200 per month
Platform fee	2% of rent

Monthly distributable cash:

```
D = R - (v + m + r + f)\,R - T
```

Where R = monthly rent, v = vacancy rate, m = management rate, r = reserve rate, f = platform fee rate, T = monthly taxes and insurance.

Worked: D = 1,400 − (0.05 + 0.08 + 0.10 + 0.02) × 1,400 − 200 = 1,400 − 350 − 200 = **$850 per month**.

Payout per token and per investor:

$$p = \frac{D}{N} \qquad \text{payout}_i = p \times t_i$$

Where N = total tokens and t = tokens held by investor i. Worked: p = 850 ÷ 15,000 = $0.0567 per token per month. A $10 investor (1 token) earns about **$0.06 per month, $0.68 per year**.

Cash yield:

$$y = \frac{12D}{\text{Total raise}}$$

Worked: 12 × 850 ÷ 150,000 = **6.8% per year**.

Sale and appreciation (simulator event):

$$\text{Proceeds per token} = \frac{V_0(1+g)^n(1-s) + \text{Reserve}}{N}$$

Where V_0 = purchase + renovation ($147,000), g = annual appreciation (default 3%, user-adjustable), n = years held, s = selling costs (6%). Worked at year 7: 147,000 × 1.03^7 × 0.94 + 3,000 = about $172,950, or **$11.53 per token**. A $10 investor would have received about $4.76 in rent plus $11.53 at sale, roughly $16.29 total. Label all of this as an estimate.

6.2 Business revenue share: sample "Corner Cuts Barbershop" (fictional)

Input	Value
Raise	$25,000
Token price	$10 (2,500 tokens)
Revenue share	5% of monthly revenue
Repayment cap	1.4x ($35,000 total)
Annual revenue	$200,000

$$P_m = \min\left(k \cdot \text{Rev}_m,\; C - \textstyle\sum \text{paid}\right) \qquad C = c \times \text{Raise}$$

Where k = revenue share rate, c = cap multiple. Worked: 0.05 × 16,667 = $833 per month, so the cap is reached in about **42 months**. Per token: $0.33 per month until $14 is repaid. If revenue drops 25%, payback stretches to about 56 months. If the business closes, unpaid

cap is lost (minus any first-loss coverage).

6.3 Neighborhood Fund

$$\text{NAV per token} = \frac{\text{Cash} + \sum_j \text{Value of holding}_j}{\text{Fund tokens outstanding}}$$

Holdings are valued at last appraisal (homes) or remaining expected payments discounted at 10% (business deals). Payouts from underlying projects flow into the fund and are distributed monthly the same way as 6.1.

6.4 Reinvesting (compounding)

When reinvest is on, each month's payout buys more tokens (fractional internally, rounded to whole tokens on-chain when enough accumulates):

$$t_{m+1} = t_m + \frac{p \times t_m}{\text{token price}}$$

6.5 Rent-to-own tokens for tenants

$$\text{Tenant tokens per month} = \frac{0.02 \times R}{\text{token price}}$$

Worked: 0.02 × 1,400 ÷ 10 = 2.8 tokens per month, about **$336 of ownership per year** for paying on time. Funded from the platform fee and reserve, never from other investors' principal.

6.6 Governance weight

$$w_{\text{community}} = 1 \times (1.5 \text{ if verified resident}) \qquad w_{\text{financial}} = \sqrt{t_i}$$

6.7 Compare mode baselines

- Savings account: balance × (1 + APY ÷ 12) each month, APY default 4% (editable).
- Cash: real value = balance ÷ (1 + inflation)^years, inflation default 3% (editable).

6.8 Platform revenue model (for the business case)

Revenue line	Rate
Raise fee (paid by the project, not investors)	3% of capital raised
Ongoing platform fee	2% of gross rent or revenue share
Marketplace trading fee	0.5% per trade
Fee discount for completed learning tracks	Up to 50% off trading fee

7. Smart contract specification

Write contracts in Solidity 0.8.x using Foundry and OpenZeppelin libraries, deploy to **Base Sepolia** for the MVP, and design them so they could later be deployed to Base mainnet after a professional audit. Keep each contract small and single-purpose.

Contract	Purpose	Key functions	Events
`IdentityRegistry`	Stores which wallets passed KYC, their tier (resident, student/alumni, general) and zip code hash. No personal data on-chain.	`register(wallet, tier, zipHash)`, `revoke(wallet)`, `isVerified(wallet)`	`Registered`, `Revoked`
`AssetToken` (one per property or business deal)	ERC-20 security token. Transfers allowed only between verified wallets and only after the lockup date. Based on ERC-3643 ideas, simplified.	`mint` (by Raise only), `transfer` (checks registry + lockup), `lockupEnds()`	`Transfer`

Contract	Purpose	Key functions	Events
`Raise`	Escrow for a funding round. Enforces tier windows, min $10, max per investor, goal and deadline. Releases to the asset's treasury on success, refunds on failure.	`invest(amount)`, `finalize()`, `refund()`, `currentTier()`	`Invested`, `Finalized`, `Refunded`
`Treasury` (one per asset)	The LLC's on-chain wallet. Holds USDC for renovations and reserve. Multisig-controlled (2 of 3: manager, OwnTheBlock, community steward).	`spend(to, amount, memo)` (multisig only), `balance()`	`Spent` with a memo string
`Distributor`	Pays holders. Uses the "dividend per share" accumulator pattern so payouts cost the same gas no matter how many holders. Holders pull with `claim()`.	`deposit(amount, reportHash)`, `claimable(wallet)`, `claim()`	`Deposited` (with IPFS hash of the expense report), `Claimed`

Contract	Purpose	Key functions	Events
`RevenueShareNote`	For business deals. Tracks cap, total repaid and status. Deposits route to the Distributor until the cap is hit, then the note closes.	`repay(amount, revenueReported)`, `remainingCap()`	`Repaid`, `CapReached`
`RulesRegistry`	Stores community rules per property: max rent increase (300 = 3%), reserve %, and the supermajority + 30-day timelock needed to change them.	`getRules(asset)`, `proposeChange()`, `executeChange()`	`RuleChanged`
`CommunityGovernor`	Proposals and voting with custom weights (section 6.6). Uses OpenZeppelin Governor with a custom vote-counting module.	`propose`, `castVote`, `execute`	`ProposalCreated`, `VoteCast`
`NeighborhoodFund`	ERC-4626-style vault holding many AssetTokens and stablecoins. Fund token = share of the vault.	`deposit`, `redeem` (after lockup), `nav()`	`Deposit`, `Withdraw`

Contract	Purpose	Key functions	Events
`Badges`	Soulbound (non-transferable) ERC-721 for completed lessons and milestones ("First Investment", "Blockchain Basics").	`award(wallet, badgeId)`	`Awarded`
`TenantRewards`	Mints asset tokens to tenants for verified on-time rent (admin attests each month).	`attestRent(tenant, month)`	`RentRewarded`
`MockUSDC` (testnet only)	Free test dollars with a faucet capped at $500 per wallet per day.	`faucet()`	`Transfer`

Security requirements:

- Use OpenZeppelin `AccessControl`, `ReentrancyGuard`, `Pausable`, and `SafeERC20`.
- Every admin action emits an event with a human-readable memo.
- 100% test coverage on Raise, Distributor and RevenueShareNote, including fuzz tests on payout math (sum of all claims never exceeds deposits).
- Upgradeability: none for the MVP. Deploy new versions and migrate if needed (simpler to reason about and explain).
- Pausing must never block refunds or claims of money already deposited.
- Run Slither static analysis and fix all high and medium findings.

Honest limits to state in the app: a smart contract cannot force a tenant to pay rent or prove a house exists. Those real-world facts are proven by the LLC documents, the property manager's monthly report (its hash stored on-chain), and independent audits.

8. Technical architecture

Build a Next.js progressive web app on Vercel, backed by Supabase, talking to Base Sepolia through Coinbase's developer tools. Before writing integration code, check the current official docs for each Coinbase product, since their APIs change often.

8.1 Stack

Layer	Choice	Why
App	Next.js (App Router), TypeScript, Tailwind CSS, shadcn/ui	One codebase, fast, installs on phones as a PWA
Wallets	Coinbase Smart Wallet or CDP Embedded Wallets via OnchainKit	Email/phone login, passkeys, no seed phrase
Chain access	wagmi + viem	Standard, typed, well documented
Gas	Coinbase Developer Platform Paymaster	Users never pay or hold ETH
Money in/out	Coinbase Onramp / Offramp (production); MockUSDC faucet (testnet)	Debit card and Apple Pay to USDC
Contracts	Solidity, Foundry, OpenZeppelin	Testing, fuzzing, deployment scripts
Indexing	Ponder (indexes contract events into Postgres)	Fast portfolio, ledger and Money Trail queries
Backend	Supabase: Postgres, Auth, Storage, Edge Functions	Free tier, row-level security
KYC	Persona (sandbox mode)	Phone camera ID + selfie
Documents	IPFS via Pinata; hash stored on-chain	Tamper-evident reports
Maps	Leaflet + OpenStreetMap	Free, mobile friendly
Animation and charts	Framer Motion, SVG, Recharts	Smooth simulator on mid-range phones
Languages	next-intl (English, Spanish)	Accessibility

Layer	Choice	Why
Notifications	Web Push	Payout alerts
Testing	Vitest (logic), Playwright (mobile viewport e2e), Foundry (contracts)	Confidence for the demo

8.2 Repository structure

```
owntheblock/
  apps/web/              Next.js app
    app/(tabs)/home, explore, simulate, learn, me
    app/project/[id]/    detail, invest, trace
    app/admin/           console
    lib/finance/         formulas from section 6
    lib/sim/             simulation engine
    lib/chain/           contract hooks, addresses, ABIs
    content/lessons/     lessons as MDX (en, es)
  contracts/             Foundry project (src, test, script)
  indexer/               Ponder config and handlers
  supabase/              migrations, seed.sql, edge functions
  docs/                  architecture, demo script, pitch
```

8.3 Data model (Postgres)

Table	Key columns
`users`	id, wallet_address, phone/email, tier, zip_code, kyc_status, language, created_at
`projects`	id, type (home / business / space / fund), name, address, lat, lng, status, raise_goal, token_price, token_address, raise_address, treasury_address, distributor_address, impact_tags[]
`project_financials`	project_id, rent_or_revenue, rates (vacancy, management, reserve, fee), taxes_insurance, rev_share_pct, cap_multiple
`monthly_reports`	project_id, month, gross, expenses JSON, reserve_added, distributed, report_ipfs_hash, deposit_tx

Table	Key columns
`investments`	user_id, project_id, amount, tokens, tx_hash, block_number (mirrored from indexer)
`payouts`	user_id, project_id, month, amount, claimed, claim_tx
`proposals` / `votes`	proposal type, text, options, weights, results, onchain_id
`pitch_ideas`	author, title, description, cost, location, upvotes, status
`lessons` / `lesson_progress` / `badges`	track, slug, completed_at, badge_token_id
`impact_metrics`	project_id, month, jobs, repairs, rent_vs_market, residents_owning
`sim_scenarios`	saved simulator configurations users can share by link

8.4 Screens and navigation

Bottom tab bar with five tabs:

1. **Home**: balance, this month's earnings, next payout, impact snapshot, "Continue lesson" card
2. **Explore**: map/list toggle, filters (type, risk, impact tag), Pitch Board
3. **Simulate**: Block Simulator (center tab, highlighted)
4. **Learn**: tracks, badges, streak
5. **Me**: portfolio, wallet, Money Trail, governance, settings, rent-to-own (tenants only)

Plus: onboarding, project detail, invest flow, trace view, proposal detail, business owner dashboard, admin console.

8.5 Design system

- **Feel**: warm, hopeful, neighborhood-proud. Real photos of people and places (placeholders in demo). Avoid a generic crypto look (no neon, no dark-mode-only).

- **Colors**: deep green for ownership and growth, warm gold for earnings, brick red for alerts, soft cream backgrounds. Do not use any university's official colors or logos, to avoid implying endorsement.
- **Type**: a friendly sans serif (Inter or similar), 16 px minimum body text.
- **Numbers**: always in dollars with cents; crypto amounts only in the "See on blockchain" layer.
- **Components**: cards with rounded corners, progress bars, bottom sheets for explainers, skeleton loaders, haptic feedback on confirm.

8.6 Seed data for the demo

All fictional, clearly labeled "Demo data":

- 3 homes (1204 Sample St funded and operating 18 months; 88 Example Ave raising, 64% funded; 310 Demo Ct proposed)
- 1 business deal (Corner Cuts Barbershop, 22 months of repayments)
- 1 laundromat on the Pitch Board in the community vote stage
- 1 Neighborhood Fund holding the above
- The six personas from section 2 with realistic histories
- 18 to 24 months of monthly reports, including one vacancy month and one roof repair, so the history shows real ups and downs

9. Compliance, safety and honesty guardrails

The MVP uses test money only; real money requires the legal setup below, and the app must say so clearly. Follow these rules in every screen and line of code.

9.1 Legal path (build the app so it fits this structure)

- Asset tokens and revenue-share notes are **securities**. Offer them under **Regulation Crowdfunding (Reg CF)** through a registered funding portal or broker-dealer partner. OwnTheBlock does not act as a portal itself in phase 1.
- Reg CF limits: an issuer can raise up to $5 million in 12 months; non-accredited investors have annual investment limits based on income and net worth; securities generally cannot be resold for 12 months. Enforce the investment limit in the invest flow and the lockup in `AssetToken`.

- Each property sits in its own LLC (Series LLC or separate LLCs) with an operating agreement that recognizes the token holder registry.
- KYC/AML on every investor. Sanctions screening through the KYC provider.
- Tax: holders receive a K-1 (LLC) or 1099; the app keeps an exportable earnings history.
- Have a securities attorney review before any real-money launch. Treat the numbers above as a starting point to verify, since rules and limits change.

9.2 Required disclosures in the app

- Persistent banner on testnet: "Demo using test money. No real dollars are invested."
- On every project and in the simulator: "Estimates, not guarantees. You can lose some or all of your investment. Past results don't predict future results."
- Risk level and specific risk factors on every project page.
- Plain-English summary of fees before every purchase.

9.3 User protection rules

- No leverage, no borrowing to invest, no margin.
- Cooling-off: the first investment above $100 requires a 24-hour wait and a reminder to keep an emergency fund.
- Spending-safety prompt: if a user invests more than 10% of their stated monthly income in a month, show a gentle check-in.
- Lessons never pay cash rewards (avoid incentivizing risky behavior).
- Anti-scam: OwnTheBlock never asks for codes or keys by text or DM; show this on the login screen.
- Data privacy: no personal data on-chain. Store only hashes and wallet addresses. Supabase row-level security on every table.

9.4 Community protection rules

- Rent increase caps and reserve rules are in `RulesRegistry` and change only by supermajority plus a 30-day delay.
- No single non-resident wallet can hold more than 10% of any asset's tokens.
- Tenants get right of first refusal on any sale.
- Governance weights favor residents (section 6.6) so outside money cannot take control.

10. Build plan and instructions to the AI agent

Build in six milestones, simulator first, so there is a demo-ready product after Milestone 2 even if later milestones slip.

10.1 How to work

1. Start by restating the plan for the current milestone in 5 to 10 bullets and list any questions. Wait for my answers if a decision is expensive to reverse.
2. Work one milestone at a time. At the end of each, run all tests, give me a short summary, the commands to run it locally, and what to check on my phone.
3. Explain every new concept in one or two plain-English sentences in your summary (I'm a finance student learning blockchain development).
4. Never commit secrets. Use `.env.local` and provide `.env.example`.
5. Prefer simple, readable code over clever code. Comment the why, not the what.
6. If a library or API has changed from what this spec says, follow the current official docs and tell me what you changed.
7. Never use real money, mainnet, or real personal data.

10.2 Milestones

#	Milestone	What gets built	Done when
1	Foundation + finance engine	Monorepo, Next.js PWA shell with 5 tabs, design system, `/lib/finance` with all section 6 formulas, seed data	Unit tests reproduce every worked example in section 6 to the cent; app installs on a phone
2	Block Simulator (sandbox)	`/lib/sim` engine, illustrated block, money flow animation, ledger strip, results counters, impact meters, controls, scenario events, story mode, compare mode	A first-time user completes story mode in under 3 minutes on a 375 px screen at 60 fps; every number has a working "How is this calculated?" link

#	Milestone	What gets built	Done when
3	Smart contracts	All section 7 contracts with Foundry tests and fuzzing, deploy scripts to Base Sepolia, verified source on the block explorer	Full test suite passes; a script runs a whole lifecycle: register → raise → finalize → deposit → claim → vote
4	Wallet, invest and payouts (testnet)	Smart wallet login, paymaster gas sponsorship, MockUSDC faucet, Explore, project detail, invest flow, portfolio, claim, Ponder indexer	A brand-new user on a phone signs up, gets test dollars, invests $10, and claims a payout without ever seeing a gas prompt
5	Transparency, governance, learning	Money Trail, transparency pages, Impact dashboard, governance, Pitch Board, Learn tracks with badges, simulator Live testnet mode	Every Money Trail step links to a real Base Sepolia transaction or an uploaded document
6	Polish and demo	Rent-to-own, business owner dashboard, admin console, Spanish, accessibility pass, Playwright e2e on mobile viewport, demo script	Lighthouse mobile score 90+ for performance and accessibility; demo runs start to finish with no errors 3 times in a row

10.3 Global acceptance criteria

- Every screen works at 375 px width and on a slow 4G network profile.
- No user ever needs ETH, a seed phrase, or a laptop.
- Every dollar figure can be traced to either a formula or an on-chain transaction.
- Simulator and testnet banners are always visible in their modes.
- All contract payout math passes fuzz tests: total claimed never exceeds total deposited.

10.4 Deliverables

- Working app deployed on Vercel with a public URL
- Contracts deployed and verified on Base Sepolia, addresses in the README

- README with setup steps a beginner can follow
- `docs/architecture.md` with a diagram of how the pieces connect
- `docs/demo-script.md` (see section 11)
- A short video-ready demo path through the app

11. Your action plan (not part of the prompt)

You can build the competition version almost entirely on free tiers in about six weeks of steady work, with the simulator done in the first two.

11.1 Accounts to set up

Account	Used for	Cost
GitHub	Code storage	Free
Claude plan that includes Claude Code	The AI builder	Paid subscription (check current pricing)
Vercel	Hosting the app	Free tier
Supabase	Database, login, file storage	Free tier
Coinbase Developer Platform	Smart wallets, paymaster, onramp, Base Sepolia access	Free for testnet
Persona (or similar KYC)	Identity check sandbox	Free sandbox
Pinata	Storing reports on IPFS	Free tier
Domain name (optional)	A custom web address	About $10 to $15 per year

You also need Node.js and Git installed on your computer to run Claude Code. Everything users touch runs on their phones.

11.2 Six-week timeline

Week	Focus	Milestones
1	Setup, finance engine, app shell	1
2	Block Simulator (your demo centerpiece)	2

Week	Focus	Milestones
3	Smart contracts on Base Sepolia	3
4	Wallets, investing, payouts	4
5	Money Trail, governance, learning, impact	5
6	Polish, demo rehearsal, pitch deck	6

If your deadline is sooner, stop after Milestone 2 plus a clickable mock of the invest flow. A great simulator with honest numbers beats a half-working full app.

11.3 Five-minute demo script for judges

1. **Hook (30 sec).** "Walk two blocks from A&T and count how many homes are owned by people who live there." Show the homeownership gap stat.
2. **Simulator story mode (90 sec).** Hand a judge your phone. Let them press Play and watch $10 travel into a house, rent come back, and the block transform.
3. **Risk honesty (30 sec).** Trigger "Roof repair" and "Tenant moves out." Show payouts dip and the reserve absorb it.
4. **Real blockchain (60 sec).** Switch to Live testnet mode. Invest $10 with Face ID, show the transaction appear on the Base Sepolia explorer in seconds, then open the Money Trail.
5. **Community impact (30 sec).** Impact dashboard: residents who are owners, rent vs market, local dollars kept.
6. **Wealth levers (30 sec).** "Your first $10 builds income, credit, knowledge and ownership at the same time." Show rent-to-own and the Learn badges.
7. **Path to real money (30 sec).** Reg CF partner, LLC structure, Greensboro pilot goals, scaling to 100+ HBCU communities.

11.4 Questions judges will likely ask

- "Isn't $0.68 a year too small to matter?" → It's the starting point. Wealth comes from stacking: regular contributions, reinvesting, rent-to-own equity, credit building and education.
- "How do you stop outside investors from taking over?" → Resident-weighted votes, 10% wallet caps, local-first access windows.

- "What happens if a business fails?" → Diversification through the Neighborhood Fund, first-loss partners, and the simulator shows this risk openly.
- "Is this legal?" → Reg CF through a registered funding portal, an LLC per asset, KYC on every user, 12-month lockup enforced in code.
- "Why blockchain instead of a normal database?" → Thousands of tiny owners, automatic payouts, public proof of every dollar, and rules no single party can quietly change.